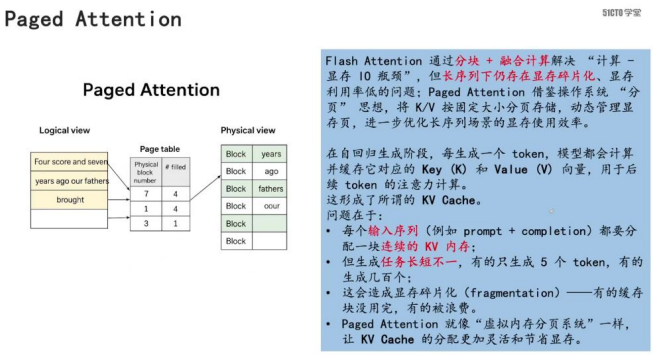
以下为AI生成的图文笔记的内容

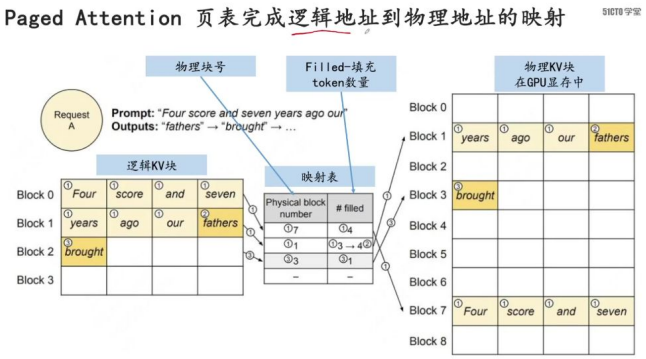
一、Paged Attention 00:00

1. Flash Attention存在的问题



-
- **IO瓶颈解决方案**：Flash Attention通过分块+融合计算的方式，将KV Cache切成小块处理，解决了计算单元与显存之间的IO瓶颈问题。
- **长序列显存问题**：在长序列场景下（如文本特别长时），会出现显存碎片化现象。不同序列长度导致显存分配不均，短序列分配的显存块未充分利用，形成"空洞"。
- **连续性需求**：自回归生成过程中，每个token的KV Cache需要连续存储以便流式多处理器顺序处理，但物理显存因频繁删除/新增数据难以保证连续性。

2. Paged Attention页表完成逻辑地址到物理地址的映射 06:03



-
- **核心思想**：借鉴操作系统分页机制，通过页表(page table)连接逻辑视图与物理存储。
- **逻辑视图**：将输入/输出序列（如"Four score and seven years ago our fathers brought..."）在逻辑内存中连续存放，每个逻辑块(block)固定大小（示例中为4个token位置）。
- **物理存储**：实际KV Cache分散存储在非连续的物理块中（如block7存"Four score and seven"，block1存"years ago our"）。
- **映射机制**：
  - **Physical block number**：标识逻辑块对应的物理块编号（如逻辑block0→物理block7）
  - **#filled**：记录物理块中实际存储的token数量（如物理block7存了4个token）
- **处理流程**：流式多处理器先读取逻辑块信息，通过页表查找物理地址，最终获取分散存储的KV Cache进行计算。

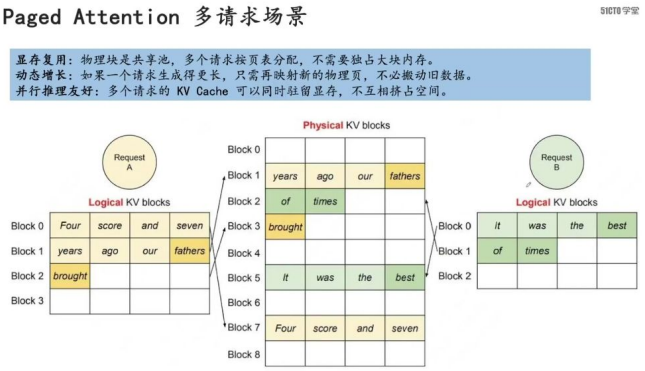
1）示例 06:10

- **具体案例**：请求"Four score and seven years ago our fathers brought..."
  - 逻辑block0：存前4个token（实际占用物理block7）

- 逻辑block1：存后续3个token+"fathers"（占用物理block1，其中前3个为输入，第4个为输出）
- 逻辑block2：存输出"brought"（占用物理block2）

- **动态扩展**：当序列变长时，只需在物理空间新增block并更新页表，无需移动已有数据。

2）多请求场景 11:31



- **显存复用**：物理块作为共享池，不同请求（如RequestA/B）通过各自页表分配物理块，避免独占大块连续显存。
- **动态增长**：单个请求序列变长时，只需映射新的物理页，保持旧数据原位不动。
- **并行优势**：多个请求的KV Cache可同时驻留显存，通过页表隔离，不会互相挤占空间。
- **资源回收**：已完成计算的物理块可释放给其他请求使用，提高显存利用率。
- **架构适配性**：特别适合Transformer的自回归生成过程，每个token的处理既保持逻辑连续性，又充分利用碎片化物理显存。

二、知识小结

主题	核心解决方案	关键技术	应用场景	创新点
Flash Attention	解决计算与显存IO瓶颈	分块计算+融合计算	GPU长序列处理	**计算资源优化**
KV Cache管理	显存碎片化处理	分页内存管理	不等长序列处理	**逻辑-物理地址映射**
Page Attention机制	保持KV Cache连续性	页表映射技术	多请求并行处理	**动态内存分配**
流式多处理器优化	顺序执行效率提升	逻辑块连续存储	Transformer架构	**硬件友好设计**

技术方案	处理对象	优势	局限性	典型应用
分块计算	长序列数据	减少IO开销	需要额外调度	**Flash Attention**
页表映射	碎片化显存	保持逻辑连续性	映射表维护成本	**Page Attention**
动态分配	多请求并发	显存高效复用	垃圾回收复杂度	**KV Cache管理**

优化维度	具体措施	效果提升	实现难度	注意事项

计算效率	分块并行处理	30-50%加速	★★★☆	**块大小调优**
显存管理	逻辑连续存储	减少60%碎片	★★★★	**页表更新频率**
硬件适配	SM单元优化	提升2倍吞吐	★★☆☆	**架构依赖性**